```python
#!/usr/bin/env python
"""
Title: Retrieval Augmented Generation for Biomedical Database with AI Language Model
Description: This script combines three approaches for Retrieval-Augmented Generation (RAG)
             using multiple AI models (MedCPT-based encoders, llama_index with OpenAI embeddings,
             and direct LLM query via llama_index). It processes PDFs, builds FAISS indices,
             performs dense retrieval, re-ranking, LLM answer generation, and evaluates results
             using ROUGE, BLEU, and RAGAS metrics.
Author: Vatsal P. Patel
Date: 12/02/25
"""

##################################################
# 0. Package Imports and Environment Setup
##################################################
import os
import shutil
import json
import csv
import numpy as np
import torch
import faiss
import PyPDF2
from dotenv import load_dotenv

# Transformers & Tokenizers
from transformers import AutoTokenizer, AutoModel, AutoModelForSequenceClassification

# Llama-index / LangChain related imports
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, StorageContext, load_index_from_storage
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.llms.openai import OpenAI as LlamaOpenAI
from llama_index.core.schema import TextNode
from llama_index.core.ingestion import IngestionPipeline
from llama_index.core.node_parser import SentenceSplitter
from langchain.vectorstores import FAISS as LC_FAISS

# Evaluation libraries
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
from rouge_score import rouge_scorer

# RAGAS evaluation
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import faithfulness, answer_correctness, context_recall,
```

```
    context_precision, answer_relevancy

# Load environment variables from .env file if available
load_dotenv()

# Set your OpenAI API key (replace with your actual key)
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "YOUR_OPENAI_API_KEY_HERE")
os.environ["OPENAI_API_KEY"] = OPENAI_API_KEY

# Global configuration parameters (adjust as needed)
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

################################################
# 1. Common Utility Functions
################################################
def extract_text_from_pdf(pdf_path):
    """Extract text from a PDF file."""
    text = ""
    with open(pdf_path, 'rb') as f:
        reader = PyPDF2.PdfReader(f)
        for page in reader.pages:
            page_text = page.extract_text()
            if page_text:
                text += page_text + "\n"
    return text

def chunk_text(text, chunk_size=500, overlap=50):
    """Chunk text into overlapping segments."""
    words = text.split()
    chunks = []
    start = 0
    while start < len(words):
        end = start + chunk_size
        chunk = words[start:end]
        if not chunk:
            break
        chunks.append(" ".join(chunk))
        start += (chunk_size - overlap)
    return chunks

################################################
# 2. Method 1: RAG using MedCPT Encoders & FAISS
################################################
def method1_pipeline(pdf_folder, output_dir):
    """
    Pipeline using MedCPT-based article/query/cross encoders for:
      - PDF text extraction and chunking
      - Document embedding and FAISS index building
      - Dense retrieval, cross-encoder re-ranking, and LLM answer generation
      - Evaluation (ROUGE, BLEU, RAGAS)
    """
    # Configuration parameters for MedCPT models
    INDEX_SAVE_PATH = os.path.join(output_dir, "faiss_index_medcpt.bin")
```

```
DOC_META_PATH = os.path.join(output_dir, "doc_metadata_medcpt.json")
MAX_ARTICLE_LENGTH = 512
MAX_QUERY_LENGTH = 64
CHUNK_SIZE = 500
OVERLAP = 50

# Model names (ensure these are accessible)
ARTICLE_ENCODER_MODEL = "ncbi/MedCPT-Article-Encoder"
QUERY_ENCODER_MODEL = "ncbi/MedCPT-Query-Encoder"
CROSS_ENCODER_MODEL = "ncbi/MedCPT-Cross-Encoder"

#####################################
# Step 2: Process PDFs and Prepare Data
#####################################
doc_metadata = []
doc_text_chunks = []
doc_id = 0

for filename in os.listdir(pdf_folder):
    if filename.lower().endswith(".pdf"):
        pdf_path = os.path.join(pdf_folder, filename)
        text = extract_text_from_pdf(pdf_path)
        chunks = chunk_text(text, chunk_size=CHUNK_SIZE, overlap=OVERLAP)
        for chunk_id, chunk in enumerate(chunks):
            doc_metadata.append({
                "doc_id": doc_id,
                "filename": filename,
                "chunk_id": chunk_id,
                "text": chunk
            })
        doc_id += 1
        doc_text_chunks.extend(chunks)

#####################################
# Step 3: Embedding Documents (Article Encoder)
#####################################
article_tokenizer = AutoTokenizer.from_pretrained(ARTICLE_ENCODER_MODEL)
article_model = AutoModel.from_pretrained(ARTICLE_ENCODER_MODEL).to(DEVICE
    )
article_model.eval()

def embed_documents(doc_chunks, batch_size=8):
    all_embeds = []
    for i in range(0, len(doc_chunks), batch_size):
        batch = doc_chunks[i:i+batch_size]
        with torch.no_grad():
            encoded = article_tokenizer(batch, truncation=True, padding=
                True, return_tensors='pt', max_length=MAX_ARTICLE_LENGTH)
            for k in encoded:
                encoded[k] = encoded[k].to(DEVICE)
            outputs = article_model(**encoded).last_hidden_state[:, 0, :]
            all_embeds.append(outputs.cpu().numpy())
    return np.vstack(all_embeds) if all_embeds else np.array([])
```

```
149
150     all_embeddings = embed_documents(doc_text_chunks)
151     print("Method1: Total embeddings shape:", all_embeddings.shape)
152
153     ######################################
154     # Step 4: Build FAISS Index
155     ######################################
156     dimension = all_embeddings.shape[1]
157     index = faiss.IndexFlatIP(dimension)  # Using inner product similarity
158     index.add(all_embeddings)
159     faiss.write_index(index, INDEX_SAVE_PATH)
160     with open(DOC_META_PATH, 'w') as f:
161         json.dump(doc_metadata, f)
162     print("Method1: FAISS index built and metadata saved.")
163
164     ######################################
165     # Step 5: Query & Re-Ranking Functions
166     ######################################
167     query_tokenizer = AutoTokenizer.from_pretrained(QUERY_ENCODER_MODEL)
168     query_model = AutoModel.from_pretrained(QUERY_ENCODER_MODEL).to(DEVICE)
169     query_model.eval()
170
171     def embed_query(queries):
172         with torch.no_grad():
173             encoded = query_tokenizer(queries, truncation=True, padding=True,
                    return_tensors='pt', max_length=MAX_QUERY_LENGTH)
174             for k in encoded:
175                 encoded[k] = encoded[k].to(DEVICE)
176             outputs = query_model(**encoded).last_hidden_state[:, 0, :]
177             return outputs.cpu().numpy()
178
179     cross_tokenizer = AutoTokenizer.from_pretrained(CROSS_ENCODER_MODEL)
180     cross_model = AutoModelForSequenceClassification.from_pretrained(
        CROSS_ENCODER_MODEL).to(DEVICE)
181     cross_model.eval()
182
183     def rerank(query, candidates):
184         pairs = [[query, candidate["text"]] for candidate in candidates]
185         with torch.no_grad():
186             encoded = cross_tokenizer(pairs, truncation=True, padding=True,
                    return_tensors="pt", max_length=512)
187             for k in encoded:
188                 encoded[k] = encoded[k].to(DEVICE)
189             logits = cross_model(**encoded).logits.squeeze(dim=1)
190         return logits.cpu().numpy()
191
192     def search_with_rerank(query, k=5):
193         query_embedding = embed_query([query])
194         scores, inds = index.search(query_embedding, k)
195         candidates = []
196         for score, ind in zip(scores[0], inds[0]):
197             entry = doc_metadata[ind]
198             entry["retrieval_score"] = float(score)
```

```
            candidates.append(entry)
        rerank_scores = rerank(query, candidates)
        for i, score in enumerate(rerank_scores):
            candidates[i]["rerank_score"] = float(score)
        candidates = sorted(candidates, key=lambda x: x["rerank_score"],
           reverse=True)
        return candidates

    ######################################
    # Step 6: LLM-based Answer Generation (Using OpenAI via llama_index
       wrapper)
    ######################################
    def get_llm_answer(query, retrieved_candidates):
        context_text = " ".join([cand["text"] for cand in retrieved_candidates
           ])
        prompt = f"""
You are a knowledgeable assistant. Use the context below to answer the
   question in one word or few.

Context:
{context_text}

Question: {query}

Answer (in one word or few):
"""
        llm = LlamaOpenAI(model="gpt-4", temperature=0)
        response = llm.complete(prompt)
        return response, context_text

    ######################################
    # Step 7: Evaluation using ROUGE & BLEU
    ######################################
    sample_queries = [
        "Which biomarker is significantly elevated in the plasma of Gaucher
           Disease Type 1 patients?",
        "What therapy reduces urinary GlcSph levels in Gaucher Disease
           patients?"
        # ... add more queries as needed
    ]
    ground_truths = [
        "Glucosylsphingosine (GlcSph).",
        "Enzyme Replacement Therapy (ERT)."
        # ... corresponding ground truths
    ]

    def evaluate_results_with_rerank(queries, ground_truths, k=3):
        rouge_scorer_instance = rouge_scorer.RougeScorer(['rouge1', 'rouge2',
           'rougeL'], use_stemmer=True)
        smoothing_function = SmoothingFunction().method4
        results = []
        for query, ground_truth in zip(queries, ground_truths):
            retrieved_results = search_with_rerank(query, k=k)
```

```
            llm_answer, context_text = get_llm_answer(query, retrieved_results)
            response_text = str(llm_answer).strip()
            rouge_scores = rouge_scorer_instance.score(ground_truth, response_text)
            bleu_score = sentence_bleu(
                [ground_truth.split()],
                response_text.split(),
                smoothing_function=smoothing_function
            )
            results.append({
                "query": query,
                "ground_truth": ground_truth,
                "response": response_text,
                "retrieved_context": context_text,
                "rouge1": rouge_scores['rouge1'].fmeasure,
                "rouge2": rouge_scores['rouge2'].fmeasure,
                "rougeL": rouge_scores['rougeL'].fmeasure,
                "bleu": bleu_score
            })
        return results

    eval_results = evaluate_results_with_rerank(sample_queries, ground_truths)
    csv_path = os.path.join(output_dir, "evaluation_metrics_medcpt.csv")
    with open(csv_path, mode='w', newline='', encoding='utf-8') as csvfile:
        writer = csv.writer(csvfile, quoting=csv.QUOTE_MINIMAL, escapechar='\\')
        writer.writerow(["Query", "Ground Truth", "Response", "Retrieved Context", "ROUGE-1", "ROUGE-2", "ROUGE-L", "BLEU"])
        for result in eval_results:
            writer.writerow([
                result["query"],
                result["ground_truth"],
                result["response"],
                result["retrieved_context"],
                f"{result['rouge1']:.4f}",
                f"{result['rouge2']:.4f}",
                f"{result['rougeL']:.4f}",
                f"{result['bleu']:.4f}"
            ])
    print(f"Method1: Evaluation metrics saved to {csv_path}")

    ######################################
    # Step 8: RAGAS Evaluation (Qualitative)
    ######################################
    data_samples = {
        'question': [result["query"] for result in eval_results],
        'answer': [result["response"] for result in eval_results],
        'contexts': [[result["retrieved_context"]] for result in eval_results],
        'ground_truth': [result["ground_truth"] for result in eval_results]
    }
    dataset = Dataset.from_dict(data_samples)
```

```
    score = evaluate(dataset, metrics=[faithfulness, answer_correctness,
        context_recall, context_precision, answer_relevancy])
    df = score.to_pandas()
    ragas_csv = os.path.join(output_dir, "ragas_evaluation_medcpt.csv")
    df.to_csv(ragas_csv, index=False)
    print(f"Method1: RAGAS evaluation scores saved to {ragas_csv}")

##################################################
# 3. Method 2: RAG using llama_index (OpenAI Embeddings + FAISS)
##################################################
def method2_pipeline(pdf_folder, output_dir):
    """
    Pipeline using llama_index to:
      - Load and chunk PDFs
      - Build a FAISS index using OpenAI embeddings
      - Retrieve documents and answer queries via OpenAI LLM
      - Evaluate responses using ROUGE, BLEU, and RAGAS metrics
    """
    # Configuration parameters
    PERSIST_DIR = os.path.join(output_dir, "storage_llama")
    DOC_META_PATH = os.path.join(output_dir, "doc_metadata_llama.json")
    INDEX_SAVE_PATH = os.path.join(output_dir, "faiss_index_llama.bin")
    EMBEDDING_MODEL = "text-embedding-ada-002"
    LLM_MODEL = "gpt-4"
    CHUNK_SIZE = 500
    OVERLAP = 50

    # Step 1: Read PDFs, chunk text, and create nodes
    documents = []
    doc_metadata = []
    doc_id = 0
    for filename in os.listdir(pdf_folder):
        if filename.lower().endswith(".pdf"):
            pdf_path = os.path.join(pdf_folder, filename)
            text = extract_text_from_pdf(pdf_path)
            splitter = SentenceSplitter(chunk_size=CHUNK_SIZE, chunk_overlap=
                OVERLAP)
            chunks = splitter.split_text(text)
            for chunk_id, chunk in enumerate(chunks):
                node = TextNode(text=chunk, id_=f"{doc_id}_{chunk_id}")
                doc_metadata.append({
                    "doc_id": doc_id,
                    "filename": filename,
                    "chunk_id": chunk_id,
                    "text": chunk
                })
                documents.append(node)
            doc_id += 1

    # Step 2: Build FAISS index with OpenAI embeddings
    storage_context = StorageContext.from_defaults()
    embedding_model = OpenAIEmbedding(model=EMBEDDING_MODEL)
    dimension = 1536  # Dimension for text-embedding-ada-002
```

```
    faiss_index = faiss.IndexFlatL2(dimension)
    # Compute embeddings for each document node
    for node in documents:
        embed = embedding_model.get_text_embedding(node.text)
        faiss_index.add(np.array(embed).reshape(1, -1))
    # Save metadata and index
    with open(DOC_META_PATH, 'w') as f:
        json.dump(doc_metadata, f)
    faiss.write_index(faiss_index, INDEX_SAVE_PATH)
    print("Method2: FAISS index built using llama_index pipeline.")

    # Step 3: Query Pipeline using llama_index LLM
    def retrieve_documents(query, top_k=3):
        query_embedding = embedding_model.get_text_embedding(query)
        scores, indices = faiss_index.search(np.array(query_embedding).reshape
           (1, -1), top_k)
        results = []
        for i, score in zip(indices[0], scores[0]):
            if i != -1:
                results.append({
                    "score": score,
                    "content": doc_metadata[i]["text"],
                    "metadata": doc_metadata[i]
                })
        return results

    def query_with_llm(query, retrieved_docs):
        context = "\n".join([doc["content"] for doc in retrieved_docs])
        prompt = f"""
You are a knowledgeable assistant. Use the context below to answer the
   question concisely in as few words as possible.
Context:
{context}

Question: {query}
Answer (in minimal words):"""
        llm = LlamaOpenAI(model=LLM_MODEL, temperature=0)
        return llm.complete(prompt)

    # Step 4: Evaluation using ROUGE and BLEU
    sample_queries = [
        "What metabolites are associated with breast cancer?"
        # Add additional queries as needed
    ]
    ground_truths = [
        "Sample ground truth answer for breast cancer metabolites."
        # Add corresponding ground truths
    ]
    rouge_scorer_instance = rouge_scorer.RougeScorer(['rouge1', 'rouge2', '
       rougeL'], use_stemmer=True)
    smoothing_function = SmoothingFunction().method4
```

```
    def evaluate_results_with_llm(queries, ground_truths, top_k=3):
        results = []
        for query, ground_truth in zip(queries, ground_truths):
            retrieved_docs = retrieve_documents(query, top_k=top_k)
            context_text = "\n".join([doc["content"] for doc in retrieved_docs
                ])
            response = query_with_llm(query, retrieved_docs)
            response_text = str(response).strip()
            rouge_scores = rouge_scorer_instance.score(ground_truth,
                response_text)
            bleu_score = sentence_bleu(
                [ground_truth.split()],
                response_text.split(),
                smoothing_function=smoothing_function,
                weights=(0.5, 0.5, 0, 0)
            )
            results.append({
                "query": query,
                "ground_truth": ground_truth,
                "response": response_text,
                "retrieved_context": context_text,
                "rouge1": rouge_scores['rouge1'].fmeasure,
                "rouge2": rouge_scores['rouge2'].fmeasure,
                "rougeL": rouge_scores['rougeL'].fmeasure,
                "bleu": bleu_score
            })
        return results

    eval_results = evaluate_results_with_llm(sample_queries, ground_truths)
    csv_path = os.path.join(output_dir, "evaluation_metrics_llama.csv")
    with open(csv_path, mode='w', newline='', encoding='utf-8') as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(["Query", "Ground Truth", "Response", "Retrieved
            Context", "ROUGE-1", "ROUGE-2", "ROUGE-L", "BLEU"])
        for result in eval_results:
            writer.writerow([
                result["query"],
                result["ground_truth"],
                result["response"],
                result["retrieved_context"],
                f"{result['rouge1']:.4f}",
                f"{result['rouge2']:.4f}",
                f"{result['rougeL']:.4f}",
                f"{result['bleu']:.4f}"
            ])
    print(f"Method2: Evaluation metrics saved to {csv_path}")

    # Step 5: RAGAS Evaluation for Method2
    data_samples = {
        'question': [res["query"] for res in eval_results],
        'answer': [res["response"] for res in eval_results],
        'contexts': [[res["retrieved_context"]] for res in eval_results],
        'ground_truth': [res["ground_truth"] for res in eval_results]
```

```
    }
    dataset = Dataset.from_dict(data_samples)
    score = evaluate(dataset, metrics=[faithfulness, answer_correctness,
       context_recall, context_precision, answer_relevancy])
    df = score.to_pandas()
    ragas_csv = os.path.join(output_dir, "ragas_evaluation_llama.csv")
    df.to_csv(ragas_csv, index=False)
    print(f"Method2: RAGAS evaluation scores saved to {ragas_csv}")

################################################
# 4. Method 3: Direct LLM Query with llama_index (Concise Responses)
################################################
def method3_pipeline(output_dir):
    """
    Pipeline that directly queries OpenAI LLM (via llama_index wrapper) for
       concise, one-word answers.
    It then evaluates the responses with ROUGE and BLEU, and performs a RAGAS
       evaluation without retrieval context.
    """
    LLM_MODEL = "gpt-4"
    TEMPERATURE = 0

    def test_llama_index_openai_llm(prompt, model=LLM_MODEL, temperature=
       TEMPERATURE):
        concise_prompt = f"{prompt}\nAnswer in one word or less:"
        llm = LlamaOpenAI(model=model, temperature=temperature)
        try:
            response = llm.complete(concise_prompt)
            return response
        except Exception as e:
            return f"Error: {e}"

    # Sample queries and ground truths for evaluation
    sample_queries = [
        "What is the capital of France?",
        "What is the chemical symbol for water?"
    ]
    ground_truths = [
        "Paris",
        "H2O"
    ]
    rouge_scorer_instance = rouge_scorer.RougeScorer(['rouge1', 'rouge2', '
       rougeL'], use_stemmer=True)
    smoothing_function = SmoothingFunction().method4

    def evaluate_responses(queries, ground_truths):
        results = []
        for query, ground_truth in zip(queries, ground_truths):
            response = test_llama_index_openai_llm(query)
            response_text = response.text.strip() if hasattr(response, "text")
                else str(response).strip()
            rouge_scores = rouge_scorer_instance.score(ground_truth,
               response_text)
```

```
            bleu_score = sentence_bleu(
                [ground_truth.split()],
                response_text.split(),
                smoothing_function=smoothing_function
            )
            results.append({
                "query": query,
                "ground_truth": ground_truth,
                "response": response_text,
                "rouge1": rouge_scores['rouge1'].fmeasure,
                "rouge2": rouge_scores['rouge2'].fmeasure,
                "rougeL": rouge_scores['rougeL'].fmeasure,
                "bleu": bleu_score
            })
        return results

    eval_results = evaluate_responses(sample_queries, ground_truths)
    csv_path = os.path.join(output_dir, "evaluation_metrics_direct_llm.csv")
    with open(csv_path, mode='w', newline='', encoding='utf-8') as csvfile:
        writer = csv.writer(csvfile)
        writer.writerow(["Query", "Ground Truth", "Response", "ROUGE-1", "
           ROUGE-2", "ROUGE-L", "BLEU"])
        for result in eval_results:
            writer.writerow([
                result["query"],
                result["ground_truth"],
                result["response"],
                f"{result['rouge1']:.4f}",
                f"{result['rouge2']:.4f}",
                f"{result['rougeL']:.4f}",
                f"{result['bleu']:.4f}"
            ])
    print(f"Method3: Evaluation metrics saved to {csv_path}")

    # RAGAS Evaluation (without retrieval context)
    data_samples = {
        'question': [res["query"] for res in eval_results],
        'answer': [res["response"] for res in eval_results],
        'contexts': [[] for _ in eval_results],
        'ground_truth': [res["ground_truth"] for res in eval_results]
    }
    dataset = Dataset.from_dict(data_samples)
    score = evaluate(dataset, metrics=[faithfulness, answer_correctness,
       context_recall, context_precision, answer_relevancy])
    df = score.to_pandas()
    ragas_csv = os.path.join(output_dir, "ragas_evaluation_direct_llm.csv")
    df.to_csv(ragas_csv, index=False)
    print(f"Method3: RAGAS evaluation scores saved to {ragas_csv}")

##############################################
# 5. Main: Run All Pipelines
##############################################
if __name__ == "__main__":
```

```
    # Define paths     adjust these as needed.
    BASE_DIR = os.getcwd()
    PDF_FOLDER = os.path.join(BASE_DIR, "sample_pdf_rag")  # Folder containing
        your PDFs
    OUTPUT_DIR = os.path.join(BASE_DIR, "rag_output")
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    print("Starting Method 1: MedCPT-based RAG pipeline...")
    method1_pipeline(PDF_FOLDER, OUTPUT_DIR)

    print("\nStarting Method 2: llama_index-based RAG pipeline...")
    method2_pipeline(PDF_FOLDER, OUTPUT_DIR)

    print("\nStarting Method 3: Direct LLM Query pipeline...")
    method3_pipeline(OUTPUT_DIR)

    print("\nAll pipelines completed. Check the output directory for CSV
       evaluation files.")
```